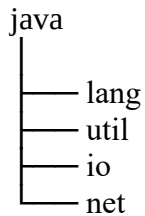


Unit 2

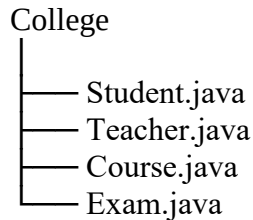
1. What is a package in Java? Mention its types and any two advantages.

A package is like a folder that contains related Java classes, interfaces, enums and sub-packages. For example,

Built-in Packages



User defined packages



Advantages:

1. Organizes the Code
2. Avoids Class Name Conflicts
3. Provides Access Protection
4. Reusability

2. How is a user-defined package created in Java?

The `package` keyword is used to create a package.

<pre>package mypackage; public class Student { public void display() { System.out.println("Student Class"); } }</pre>	<pre>import mypackage.Student; public class Main { public static void main(String[] args) { Student s = new Student(); s.display(); } }</pre>
---	---

Here `mypackage` is the user defined package.

3. What is the purpose of the import statement? Give an example.

The `import` statement allows a class or interface from another package to be used without specifying its complete package name.

```
import java.util.Scanner;
class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}
```

Here, `Scanner` is imported from `java.util`

4. What is an interface in Java? Give its syntax.

An interface in Java is a blueprint of a class that specifies **what a class must do, but not how it does it**.

```
interface Printable {
    void print();
}
```

Here `Printable` is the interface name

5. How does a class implement an interface? Give an example.

A class uses the `implements` keyword to implement an interface.

```
interface Animal {
    void sound(); //public static void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog Barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
Dog Barks
```

6. Can a class implement multiple interfaces? Explain with an example.

Yes. Java supports implementing multiple interfaces.

```
interface A {
    void show();
}
interface B {
    void display();
}
class Test implements A, B {
    public void show() {
```

```

        System.out.println("Show");
    }
    public void display() {
        System.out.println("Display");
    }
}

```

7. What is interface inheritance?

Interface inheritance means **one interface inherits the members of another interface** using the **extends** keyword.

```

interface A {
    void show();
}

interface B extends A {
    void display();
}

```

Here, interface B inherits show() from interface A.

8. What is an exception? Why is exception handling required?

An **exception** is an abnormal condition that occurs during program execution.

Exception handling is required to:

1. Detect runtime errors.
2. Handle errors without abruptly terminating the program.
3. Maintain normal program flow.

9. List the five keywords used in Java exception handling.

The five exception-handling keywords are:

1. try – contains code that may cause an exception.
2. catch – handles the exception.
3. finally – executes after try/catch.
4. throw – explicitly throws an exception.
5. throws – declares exceptions that a method may throw.

10. Differentiate between throw and throws

throw	throws
Used to explicitly throw an exception.	Used to declare exceptions.
Used inside a method/block.	Used in the method declaration.
Throws one exception object at a time.	Can declare multiple exceptions.
<pre> void test() throws IOException, ArithmeticException, ClassNotFoundException { // statements } </pre>	

11. Write a Java code fragment to handle `ArithmeticException`

```
try {  
    int a = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

12. What is a multiple catch block? Give an example.

A multiple catch block is used to handle different types of exceptions separately.

```
try {  
    int a = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Arithmetic Error");  
}  
catch (Exception e) {  
    System.out.println("Other Error");  
}  
Output:  
Arithmetic Error
```

13. What is a nested try statement?

A try block placed inside another try block is called a nested try statement.

```
try {  
    try {  
        int x = 10 / 0;  
    }  
    catch (ArithmeticException e) {  
        System.out.println("Inner exception");  
    }  
}  
catch (Exception e) {  
    System.out.println("Outer exception");  
}
```

14. Differentiate between checked and unchecked exceptions.

Checked Exception	Unchecked Exception
Checked at compile time.	Occurs during runtime.
Related to operations that may cause exceptions and need to be checked by the compiler.	Usually caused by programming mistakes.
Example: <code>IOException</code>	Example: <code>ArithmeticException</code>

15. What is a user-defined exception? Give an example.

A user-defined exception (also called a custom exception) is an exception created by the programmer to handle application-specific errors.

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}
public class Main {
    static void checkAge(int age) throws InvalidAgeException {
        if(age < 18)
            throw new InvalidAgeException("Not Eligible to Vote");
        else
            System.out.println("Eligible to Vote");
    }
    public static void main(String args[]) {
        try {
            checkAge(16);
        }
        catch(InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
Not Eligible to Vote
```

16. What is the purpose of the finally block?

Always executes whether an exception occurs or not.

```
public class Test {
    public static void main(String[] args) {

        try {
            int x = 10 / 0;
        }
        catch (ArithmeticException e) {
            System.out.println("Error occurred");
        }
        finally {
            System.out.println("Finally block");
        }
    }
}
```

17. What is multithreading? Mention any two advantages

Multithreading is the concurrent execution of multiple threads within a program.

Advantages:

1. Better CPU utilization.
2. Improved program responsiveness.
3. Allows multiple tasks to execute concurrently.
4. Can improve application performance.

18. What are the two ways of creating a thread in Java?

Extending the Thread class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
    public static void main(String args[]) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Thread is running

Implementing the Runnable interface

```
class MyRunnable implements Runnable {  
  
    public void run() {  
        System.out.println("Runnable Thread");  
    }  
  
    public static void main(String args[]) {  
  
        MyRunnable obj = new MyRunnable();  
  
        Thread t = new Thread(obj);  
  
        t.start();  
    }  
}
```

Runnable Thread

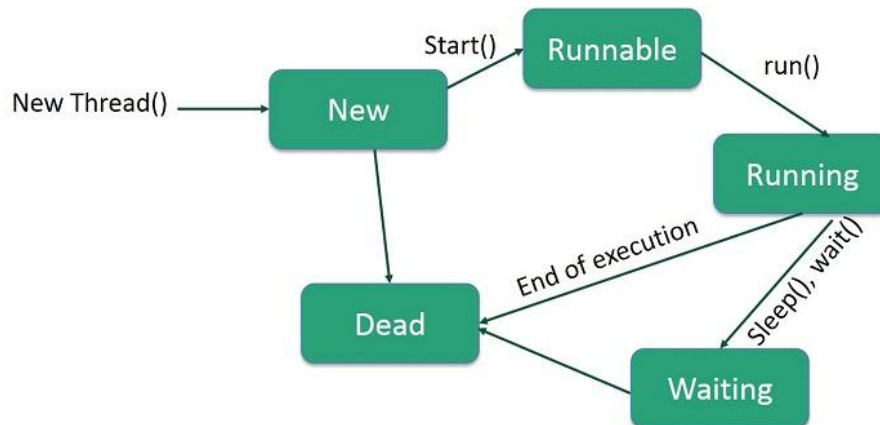
19. Differentiate between start() and run() methods.

start ()	run ()
Starts a new thread.	Contains the task executed by the thread.
Creates a separate execution path.	Executes like a normal method if called directly.
Internally invokes run () .	Does not create a new thread by itself.

20. List the major states in the Java thread life cycle.

The major thread states are:

1. **NEW** – thread is created but not started.
2. **RUNNABLE** – thread is ready/running.
3. **BLOCKED** – waiting to acquire a lock.
4. **WAITING** – waiting indefinitely.
5. **TIMED_WAITING** – waiting for a specified period.
6. **TERMINATED** – thread has completed execution.



21. What is thread priority? Mention its range.

Thread priority indicates the relative importance of a thread to the thread scheduler.

Thread.MIN_PRIORITY // 1

Thread.NORM_PRIORITY // 5

Thread.MAX_PRIORITY // 10

22. What is an enumeration in Java? Give an example.

enum is a **special data type** used to define a **fixed set of constant values** in a **type-safe way**.

```
enum TrafficLight {  
    RED, GREEN, YELLOW;  
}  
public class Test{  
  
    public static void main(String[] args){  
  
        TrafficLight x = TrafficLight.RED;  
        System.out.println(x);  
    }  
}  
RED
```

23. What is autoboxing and unboxing? Give examples.

Autoboxing is the automatic conversion of a primitive value into its corresponding wrapper object.

```
int x = 10;
```

```
Integer obj = x;
```

Unboxing is the automatic conversion of a wrapper object into its corresponding primitive value.

```
Integer obj = 20;
```

```
int x = obj;
```

24. What is the purpose of the `isAlive()` method in Java?

The `isAlive()` method is used to check whether a thread is currently running.

It returns:

- `true` – if the thread is alive.
- `false` – if the thread has not started or has completed.

```
public class Main extends Thread {
    public static int amount = 0;

    public static void main(String[] args) {
        Main obj = new Main();
        obj.start();
        // Wait for the thread to finish
        while(obj.isAlive()) {
            System.out.println("Waiting...");
        }
        System.out.println("Main: " + amount);
        amount++;
        System.out.println("Main: " + amount);
    }
    public void run() {
        amount++;
    }
}
Waiting...
Waiting...
Main: 1
Main: 2
OR
Waiting...
Main: 1
Main: 2
```


25. What are the wrapper classes in Java? Give any three examples.

Wrapper classes are classes that represent primitive data types as objects.

Primitive	Wrapper Class
int	Integer
double	Double
char	Character
boolean	Boolean

26. What is the difference between enum and class in Java?

enum	class
Used to define a fixed set of constants.	Used to define objects and their behavior.
Objects/values are predefined.	Objects can generally be created as needed.
Declared using the <code>enum</code> keyword.	Declared using the <code>class</code> keyword.